

Debugbar for Laravel (Integrates PHP Debug Bar)

MIT license

15.3k stars 1.5k forks Activity

Star

Watch

Code Issues 44 Pull requests 35 Actions Security Insights

master

erikn69 (fix) upgrade find view from hash (#1415) ...

3 weeks ago 964

View code

readme.md

Debugbar for Laravel

Unit Tests passing license MIT stable v3.8.1 downloads 57.57 M Powered By Fruitcake

This is a package to integrate [PHP Debug Bar](#) with Laravel. It includes a ServiceProvider to register the debugbar and attach it to the output. You can publish assets and configure it through Laravel. It bootstraps some Collectors to work with Laravel and implements a couple custom DataCollectors, specific for Laravel. It is configured to display Redirects and (jQuery) Ajax Requests. (Shown in a dropdown) Read [the documentation](#) for more configuration options.

Messages 4 Timeline Exceptions Views 0 Route Queries 2 Mails Auth Gate Session Request GET hello 11.23MB 241.02ms 7.2.25

```
array:2 [
  "name" => "Barry vd. Heuvel"
  "email" => "barryvdh@gmail.com"
]
```

INFO

Error! ERROR

Watch out... WARNING

Another message MYLABEL

MYLABEL WARNING ERROR INFO

Q

Note: Use the DebugBar only in development. It can slow the application down (because it has to gather data). So when experiencing slowness, try disabling some of the collectors.

This package includes some custom collectors:

- QueryCollector: Show all queries, including binding + timing
- RouteCollector: Show information about the current Route.

- ViewCollector: Show the currently loaded views. (Optionally: display the shared data)
- EventsCollector: Show all events
- LaravelCollector: Show the Laravel version and Environment. (disabled by default)
- SymfonyRequestCollector: replaces the RequestCollector with more information about the request/response
- LogsCollector: Show the latest log entries from the storage logs. (disabled by default)
- FilesCollector: Show the files that are included/required by PHP. (disabled by default)
- ConfigCollector: Display the values from the config files. (disabled by default)
- CacheCollector: Display all cache events. (disabled by default)

Bootstraps the following collectors for Laravel:

- LogCollector: Show all Log messages
- SwiftMailCollector and SwiftLogCollector for Mail

And the default collectors:

- PhpInfoCollector
- MessagesCollector
- TimeDataCollector (With Booting and Application timing)
- MemoryCollector
- ExceptionsCollector

It also provides a facade interface (`Debugbar`) for easy logging Messages, Exceptions and Time

Installation

Require this package with composer. It is recommended to only require the package for development.

```
composer require barryvdh/laravel-debugbar --dev
```

Laravel uses Package Auto-Discovery, so doesn't require you to manually add the ServiceProvider.

The Debugbar will be enabled when `APP_DEBUG` is `true` .

If you use a catch-all/fallback route, make sure you load the Debugbar ServiceProvider before your own App ServiceProviders.

Laravel without auto-discovery:

If you don't use auto-discovery, add the ServiceProvider to the providers array in config/app.php

```
Barryvdh\Debugbar\ServiceProvider::class,
```

If you want to use the facade to log messages, add this to your facades in app.php:

```
'Debugbar' => Barryvdh\Debugbar\Facades\Debugbar::class,
```

The profiler is enabled by default, if you have APP_DEBUG=true. You can override that in the config (debugbar.enabled) or by setting DEBUGBAR_ENABLED in your .env . See more options in config/debugbar.php You can also set in your config if you want to include/exclude the vendor files also (FontAwesome, Highlight.js and jQuery). If you already use them in your site, set it to false. You can also only display the js or css vendors, by setting it to 'js' or 'css'. (Highlight.js requires both css + js, so set to true for syntax highlighting)

Copy the package config to your local config with the publish command:

```
php artisan vendor:publish --provider="Barryvdh\Debugbar\ServiceProvider"
```

Laravel with Octane:

Make sure to add LaravelDebugbar to your flush list in config/octane.php .

```
'flush' => [
    \Barryvdh\Debugbar\LaravelDebugbar::class,
],
```

Lumen:

For Lumen, register a different Provider in bootstrap/app.php :

```
if (env('APP_DEBUG')) {
    $app->register(Barryvdh\Debugbar\LumenServiceProvider::class);
}
```

To change the configuration, copy the file to your config folder and enable it:

```
$app->configure('debugbar');
```

Usage

You can now add messages using the Facade (when added), using the PSR-3 levels (debug, info, notice, warning, error, critical, alert, emergency):

```
Debugbar::info($object);
Debugbar::error('Error!');
Debugbar::warning('Watch out...');
Debugbar::addMessage('Another message', 'mylabel');
```

And start/stop timing:

```
Debugbar::startMeasure('render','Time for rendering');
Debugbar::stopMeasure('render');
Debugbar::addMeasure('now', LARAVEL_START, microtime(true));
Debugbar::measure('My long operation', function() {
    // Do something...
});
```

Or log exceptions:

```
try {
    throw new Exception('foobar');
} catch (Exception $e) {
    Debugbar::addThrowable($e);
}
```

There are also helper functions available for the most common calls:

```
// All arguments will be dumped as a debug message
debug($var1, $someString, $intValue, $object);

// `$collection->debug()` will return the collection and dump it as a debug message. Like `$co
collect([$var1, $someString])->debug();

start_measure('render','Time for rendering');
stop_measure('render');
add_measure('now', LARAVEL_START, microtime(true));
measure('My long operation', function() {
    // Do something...
});
```

If you want you can add your own DataCollectors, through the Container or the Facade:

```
Debugbar::addCollector(new DebugBar\DataCollector\MessagesCollector('my_messages'));  
//Or via the App container:  
$debugbar = App::make('debugbar');  
$debugbar->addCollector(new DebugBar\DataCollector\MessagesCollector('my_messages'));
```

By default, the Debugbar is injected just before `</body>` . If you want to inject the Debugbar yourself, set the config option 'inject' to false and use the renderer yourself and follow

<http://phpdebugbar.com/docs/rendering.html>

```
$renderer = Debugbar::getJavascriptRenderer();
```

Note: Not using the auto-inject, will disable the Request information, because that is added After the response. You can add the default_request datacollector in the config as alternative.

Enabling/Disabling on run time

You can enable or disable the debugbar during run time.

```
\Debugbar::enable();  
\Debugbar::disable();
```

NB. Once enabled, the collectors are added (and could produce extra overhead), so if you want to use the debugbar in production, disable in the config and only enable when needed.

Twig Integration

Laravel Debugbar comes with two Twig Extensions. These are tested with [rcrowe/TwigBridge](#) 0.6.x

Add the following extensions to your TwigBridge config/extensions.php (or register the extensions manually)

```
'Barryvdh\Debugbar\Twig\Extension\Debug',  
'Barryvdh\Debugbar\Twig\Extension\Dump',  
'Barryvdh\Debugbar\Twig\Extension\Stopwatch',
```

The Dump extension will replace the [dump function](#) to output variables using the DataFormatter. The Debug extension adds a `debug()` function which passes variables to the Message Collector, instead of showing it directly in the template. It dumps the arguments, or when empty; all context variables.

```
{{ debug() }}  
{{ debug(user, categories) }}
```

The Stopwatch extension adds a [stopwatch tag](#) similar to the one in Symfony/Silex Twigbridge.

```
{% stopwatch "foo" %}  
    ...some things that gets timed  
{% endstopwatch %}
```

Releases 66

 **Fix font + restore btn styling** Latest
on Feb 21

[+ 65 releases](#)

Sponsor this project

 **barryvdh** Barry vd. Heuvel



 <https://fruitcake.nl>

[Learn more about GitHub Sponsors](#)

Packages

No packages published

Used by 98.5k

        [+ 98,451](#)

Contributors 185



[+ 174 contributors](#)

Languages

 PHP 75.0%  CSS 17.7%  JavaScript 7.3%